

# Assignment 2- Case Study

## Stage 3 Lab Activities

### SmartCare Domain Modelling

---

AI OFF -> AI ON -> COMPARE -> VERIFY | 1 hour

#### **A - Requirements Review**

##### **Nouns**

Patient

Practitioner

Appointment

Record

Status

History log

Profile

Schedule

##### **Verbs**

Create

Save

Search

Prevent

Update

Retain/keep

Display

Edit

View

Block

## **Business Rules**

Prevent the creation of overlapping bookings for a single practitioner. Explicitly label cancelled appointments in the history rather than deleting them.

## **B - Candidate Classes**

### **Patient**

Supported by FR-01, FR-02, FR-06, and FR-09. State: name, ID, contact info. Behaviour: should retrieve history log.

### **Practitioner**

Supported by FR-07 and FR-08. State: profile details. Behaviour: retrieve daily schedule.

### **Appointment**

Supported by FR-03, FR-04, FR-05, FR-10, and FR-11. State: date, time slot, status (scheduled, completed, cancelled).

## **C - CRC Cards**

### **Patient**

Responsibilities: Knows personal details (Name, ID, Contact Info), knows past appointment history.

Collaborators: Appointment.

### **Practitioner**

Responsibilities: Knows practitioner details like their profile, knows daily schedule of appointments.

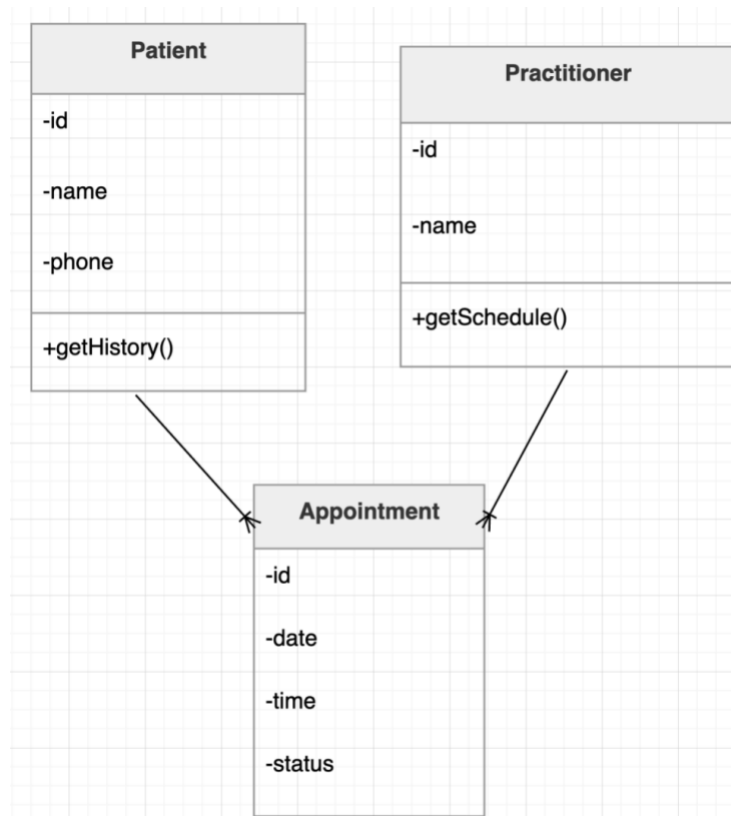
Collaborators: Appointment.

### **Appointment**

Responsibilities: Knows scheduled date, time, and status. Links a booking to a patient and practitioner.

Collaborators: Patient and practitioner.

## D - UML Model



## E - AI Design Review

**AI Suggestion 1:** Add a Status attribute to the Appointment class, to track if it is scheduled, completed, or cancelled (supported by FR-05).

**AI Suggestion 2:** Create a PatientManager class to handle searching for patients (supported by FR-02).

**AI Suggestion 3:** Create a NotificationManager class to handle sending SMS reminders.

## F - Compare and Decide

### Accepted

I accepted the suggestion to add a status attribute to the Appointment class. The reason is that FR-05 explicitly requires the system to update and track whether an appointment is scheduled, completed, or cancelled.

### Modified

I modified the suggestion for a PatientManager class. While FR-02 requires patient searching, adding a manager class violates NFR-01, which states the architecture must remain small and maintainable. I modified this by rejecting the new class and keeping search as a general system action.

### Rejected

I rejected the NotificationManager class. There is no requirement ID for this, and the client brief explicitly states that SMS integrations are out of scope.

## G - Python Skeletons

### class Patient:

```
def __init__(self, id, name, contactInfo):
```

```
    self.id = id
```

```
    self.name = name
```

```
    self.contactInfo = contactInfo
```

```
    self.appointments = []
```

```
def getHistory(self):
```

```
    return self.appointments
```

### class Practitioner:

```
def __init__(self, id, name):
```

```
    self.id = id
```

```
    self.name = name
```

```
    self.appointments = []
```

```
def getSchedule(self):  
  
    return self.appointments
```

#### **class Appointment:**

```
def __init__(self, id, date, time, status, patient, practitioner):  
  
    self.id = id  
  
    self.date = date  
  
    self.time = time  
  
    self.status = status  
  
    self.patient = patient  
  
    self.practitioner = practitioner
```

## **H - Consistency Check**

The Python code matches the UML model and the FRs perfectly. The Patient and Practitioner classes both have lists (`self.appointments = []`) to store multiple appointments, which correctly represents the 1-to-many relationship we drew in the diagram to support historical logs and daily schedules.

## **Reflection**

The hardest modelling decision was figuring out how to link Patients, Practitioners, and Appointments without creating a massive "Clinic" class to control everything. The AI over-designed the system by suggesting complex controller classes like NotificationManager and PatientManager. I rejected these ideas because they are software code details, not real-world domain concepts, and they violate NFR-01, which requires the architecture to remain small and maintainable. The evidence that supported my final choice to use only three simple classes was the main scope in the client brief, which strictly asked for a small software system for managing patients, practitioners, and appointments.